

EventGraph 外部 Agent 写入架构设计

更新日期：2026-06-30

实现状态：核心闭环已落地。当前系统已经取消内置 EventGraph agent 路线，改为外部 agent 通过受控 API 提交 EventGraph patch / change request；系统完成校验、审计、人审、Graph Core 写入和版本化。正式入口为 `http://127.0.0.1:5001`。

本文取代上一版以内置 Agent 为中心的 EventGraph SOP 设计。新的方向是：

取消内置 EventGraph agent

- > EventGraph 提供稳定读写能力
- > 外部 agent 通过受控 API 读取、研究、提交修改
- > 人类或受信规则审核后写入正式图谱
- > 全流程保留 `run / step / audit / version trace`

核心变化：

- 不再建设 `internal_agent` 调查队列。
- 不再让系统内部维护一个 LLM agent runner。
- 外部 agent 是唯一的智能研究和修改请求来源。
- 系统仍负责采集、异常计算、权限、校验、审计、版本和回滚。
- 正式 EventGraph 仍不能被无审计地直接覆盖。

当前已实现：

- `POST /api/agent/event-graph/patches/validate`：校验 patch、识别风险、规范化 patch、返回 target refs。
- `POST /api/agent/event-graph/change-requests`：外部 agent 提交变更请求。
- `GET /api/agent/event-graph/change-requests` 和 `/request_id`：查询变更请求。
- `POST /api/event-graph/change-requests/{request_id}/approve|reject|request-changes|apply`：人类审阅和应用。
- Graph Core 正式对象：`event`、`finance`、`edge`、`expression`。
- 版本化：每类对象都有 `current table` 和 `version table`，并记录 `before / after / patch item / request / run / actor`。
- 支持动作：`event_create`、`event_update`、`event_archive`、`event_merge`、`finance_create`、

- finance_update、finance_archive、edge_create、edge_update、edge_delete、finance_mapping_create、expression_create、expression_update、expression_archive。
- event_merge 会把 source event 标记为 MERGED，目标 event 升版本，并重定向相关 edge；merge 后产生自环的 edge 会自动归档。
- EventGraph 页面会合入 Graph Core 对象，详情面板可加载 Version History。

1. 新定位

EventGraph 是一套可追溯知识库，不是 agent 本身。

系统边界重新划分为：

角色	职责	不做什么
System	采集 observation、计算 anomaly、提供查询、校验 proposal、写版本、审计	不做 LLM 调查推理
External Agent	查询图谱、研究事件、生成变更请求、解释证据、必要时提交策略草案	不绕过权限、预算、审核和审计
Human	审核、修改、批准、拒绝、回滚、配置权限	不直接手改数据库
Graph Core	保存正式 Event / Finance / Edge / Expression	不保存临时搜索过程

推荐主路径：

```
Observation / Anomaly / Derived Preview
-> External Agent Research
-> Change Request / Proposal
-> Validation
-> Human Review 或 Trusted Rule Review
-> Graph Core Version
```

这比内置 agent 方案更简单，也更适合当前项目形态：Codex、Claude、独立脚本或任何外部 agent 都走同一套 API，不需要把 LLM 调度逻辑塞进 Flask 服务。

2. 数据层级保留但简化

仍保留数据层级，但去掉内置 investigation 层。

Observation

- > Anomaly
- > Candidate Hint
- > External Research
- > Change Request / Proposal
- > Review
- > Graph Core

2.1 Observation

系统采集到的原始材料：

```
news_observation
market_observation
official_announcement
binance_snapshot
polymarket_snapshot
```

Observation 永远可追溯，不因后续判断变化而删除。

2.2 Anomaly

确定性程序计算出的市场异常：

```
POLYMARKET_PROBABILITY_JUMP
POLYMARKET_VOLUME_SPIKE
BINANCE_PRICE_SHOCK
BINANCE_VOLUME_SPIKE
BINANCE_FUNDING_ANOMALY
```

Anomaly 是证据或触发线索，不是正式 Event。

2.3 Candidate Hint

取消内置 agent 后，Candidate 不再是“内置调查任务”，而是外部 agent 可读取的候选提示。

可以很轻：

```
candidate_hints
  hint_id
  hint_type = MATCHED_NEWS_MARKET | UNEXPLAINED_MARKET_MOVE | NEWS_WITHOUT_MARKET_REACTION
  title
  summary
  priority
  evidence_refs_json
  status = OPEN | USED | DISMISSED | EXPIRED
  created_at_utc
```

如果暂时不建表，也可以由 `/api/agent/event-graph` 的 `derived preview` 和 `anomaly` 查询结果承担。

2.4 External Research

外部 agent 的研究过程不落入业务表，只通过：

```
agent_runs
agent_run_steps
agent_audit_events
```

记录。

agent 自己可以把详细推理保存在 `change request` 的 `rationale`、`evidence_summary` 和 `research_trace_json` 里。

2.5 Change Request / Proposal

这是新架构最核心的写入口。

外部 agent 不直接写 `events` / `edges` / `finance_nodes`，而是提交：

```
event_create
event_update
event_merge
event_archive
finance_create
finance_update
edge_create
edge_update
edge_delete
expression_create
```

这些请求经过系统校验和人工审核后才进入 Graph Core。

3. 写入模式

建议支持三个模式，从安全到高效逐步开放。

3.1 Proposal Only

默认模式。

```
external agent
-> POST /api/agent/event-graph/change-requests
-> status=PENDING
-> human review
-> apply
```

适合所有高影响修改：

- 新增正式 Event。
- 新增因果或影响 Edge。
- Merge Event。
- Archive Event。
- 修改 confidence / mechanism。

3.2 Trusted Low Risk Apply

可选模式，只给受信 agent 和低风险 action。

```
external agent
-> POST /api/agent/event-graph/patches/apply
-> system validation
-> auto snapshot
-> apply
-> audit
```

允许范围建议只包括：

- 添加 event alias。
- 添加 observation link。
- 修正文案、source URL、tag。
- 更新非交易性 metadata。

不允许自动 apply：

- 新建影响 Edge。
- 修改 DIRECTLY_PRICES。
- Merge / Archive。
- 修改高置信度 verified object。

3.3 Dry Run Diff

所有写接口都应支持：

```
{
  "dry_run": true
}
```

返回：

```
validated
warnings
diff
affected_objects
risk_level
requires_human_review
```

这样外部 agent 可以先让系统帮它检查，再决定是否提交。

4. 外部 Agent 标准 Workflow

取消内置 Agent 后，外部 agent workflow 成为主工作流。

4.1 EXT_EVENT_RESEARCH

目标：研究某个事件，不提交修改。

```
GET /api/agent/event-graph?q=<topic>
GET /api/agent/event-graph/events?q=<topic>
GET /api/agent/event-graph/observations?q=<topic>
GET /api/agent/event-graph/anomalies?q=<topic>
GET /api/agent/event-graph/core/events?q=<topic>
```

输出：

事件摘要
已有图谱对象
证据来源
可能缺失的 Event / Edge / Finance
是否值得提交 change request

4.2 EXT_GRAPH_CHANGE_REQUEST

目标：提交图谱修改请求。

```
GET /api/agent/capabilities
GET /api/agent/event-graph/core/search
POST /api/agent/event-graph/patches/validate
POST /api/agent/event-graph/change-requests
GET /api/agent/event-graph/change-requests/{request_id}
```

agent 必须带：

```
{
  "run_id": "run_external_xxx",
  "workflow_id": "EXT_GRAPH_CHANGE_REQUEST",
  "actor_type": "agent",
  "actor_id": "agent_strategy_assistant",
  "change_type": "edge_create",
  "reason": "...",
  "evidence_summary": "...",
  "proposed_changes": {}
}
```

4.3 EXT_EVENT_TO_STRATEGY_DRAFT

目标：基于 EventGraph 生成策略草案。

流程：

```
query EventGraph
  -> inspect Finance mappings
  -> read market data
  -> draft strategy
  -> risk.check
  -> simulate
  -> submit approval
```

图谱修改和策略草案应分开：

- 图谱改动走 `event.graph.change_request`。
- 策略草案走 `strategy.draft.*` 和 `strategy.submit`。

不要让一个接口同时“改知识”和“建策略”。

5. API 设计

当前已有：

```
GET /api/agent/event-graph
GET /api/agent/event-graph/events
GET /api/agent/event-graph/observations
POST /api/agent/event-graph/news/search
POST /api/agent/event-graph/change-requests
GET /api/agent/event-graph/change-requests
```

当前 Graph Core 读接口：

5.1 Graph Core Read

```
GET /api/agent/event-graph/core
GET /api/agent/event-graph/core/events
GET /api/agent/event-graph/core/finance
GET /api/agent/event-graph/core/edges
GET /api/agent/event-graph/core/expressions
GET /api/agent/event-graph/core/versions?object_type=event|finance|edge|expression&object_id=<id>
```

用于读取正式图谱当前态和版本历史。 /core/versions 会返回

before、after、patch_item、request_id、run_id、actor 和 version_number。

5.2 Preview / Evidence Read

```
GET /api/agent/event-graph/preview
GET /api/agent/event-graph/observations
GET /api/agent/event-graph/anomalies
GET /api/agent/event-graph/candidate-hints
```

用于发现和研究。

5.3 Patch Validation

```
POST /api/agent/event-graph/patches/validate
```

请求：

```

{
  "run_id": "run_xxx",
  "workflow_id": "EXT_GRAPH_CHANGE_REQUEST",
  "patch": {
    "items": [
      {
        "action": "edge_create",
        "source_id": "evt_xxx",
        "target_id": "fin_oil",
        "relation_class": "IMPACT",
        "relation_type": "POSITIVE_IMPACT",
        "confidence": 0.68,
        "mechanism": "...",
        "evidence_refs": []
      }
    ]
  }
}

```

返回:

```

{
  "valid": true,
  "risk_level": "medium",
  "requires_human_review": true,
  "warnings": [],
  "diff": {},
  "normalized_patch": {}
}

```

5.4 Change Request

POST /api/agent/event-graph/change-requests

GET /api/agent/event-graph/change-requests

GET /api/agent/event-graph/change-requests/{request_id}

change_request 已从轻量 payload 升级为结构化 proposal:

request_id
run_id
workflow_id
requester_type
requester_id
change_type
status = DRAFT | PENDING | NEEDS_CHANGES | APPROVED | REJECTED | APPLIED | SUPERSEDED
risk_level
title
summary
reason
evidence_summary
patch_json
validation_json
created_at_utc
updated_at_utc
reviewed_at_utc
reviewer_id
review_note
applied_at_utc
applied_by_type
applied_by_id
apply_error_json

5.5 Review / Apply

Human-only:

```
POST /api/event-graph/change-requests/{request_id}/approve
POST /api/event-graph/change-requests/{request_id}/reject
POST /api/event-graph/change-requests/{request_id}/request-changes
POST /api/event-graph/change-requests/{request_id}/apply
```

Trusted rule 已开放为 Settings 控制项。默认 manual 模式下，Graph Core apply 仍通过 human/admin 审阅路径：

```
POST /api/event-graph/change-requests/{request_id}/apply
```

当 Settings 中 `agent_policy.event_graph_approval.mode=trusted_all` 时，外部 agent 提交的 change request 在通过 patch validation 后，可由 system actor 自动 approve/apply，并写入审计和版本历

史。 `trusted_low_risk` 模式保留给低风险动作；当前自动 `apply` 仍必须检查 `capability`、`risk_level`、`item` 数量、`confidence`、证据要求和当前 `apply` 引擎支持的 `action allowlist`。

6. 数据库设计

6.1 保留现有表

继续使用：

```
event_graph_observations
event_graph_events
event_graph_event_observations
event_graph_refresh_runs
agent_event_change_requests
agent_runs
agent_run_steps
agent_audit_events
```

但 `agent_event_change_requests` 需要升级。

6.2 change request 表升级

当前：

```
request_id
change_type
requester
status
payload_json
created_at_utc
reviewed_at_utc
review_note
```

建议新增字段：

run_id
workflow_id
requester_type
requester_id
title
summary
reason
evidence_summary
risk_level
patch_json
validation_json
target_refs_json
updated_at_utc
reviewer_type
reviewer_id
applied_at_utc
applied_snapshot_id
error_json

向后兼容：旧 payload_json 仍保留，作为 raw payload。

6.3 Graph Core

正式 Graph Core 已新增：

graph_events
graph_event_versions

graph_finance_nodes
graph_finance_versions

graph_edges
graph_edge_versions

graph_expressions
graph_expression_versions

正式对象必须带：

```
origin_request_id
origin_run_id
origin_actor_id
verification_status
current_version
created_at_utc
updated_at_utc
```

7. 权限模型

外部 agent 能力拆开：

```
event.read
event.news.search
event.graph.change_request
event.graph.patch.validate
event.graph.apply_low_risk
event.graph.review.read
```

默认允许：

```
event.read
event.news.search
event.graph.change_request
event.graph.patch.validate
```

默认禁止：

```
event.graph.apply_low_risk
event.graph.review.approve
event.graph.core.direct_write
```

Human/admin 才能：

```
event.graph.review.approve
event.graph.review.reject
event.graph.review.apply
```

必须后端强制，不依赖前端隐藏按钮。

8. 校验规则

任何 patch 入库前必须验证：

- action 是否合法。
- target 是否存在或允许 create。
- source / target 节点类型是否匹配。
- relation_class 与 relation_type 是否匹配。
- MAPPING edge 方向是否固定。
- confidence 是否在 0 到 1。
- high impact edge 是否有 mechanism、evidence、alternative_explanations。
- merge/archive 是否保留回滚信息。
- 外部 agent 是否具备对应 capability。
- 是否需要 human review。

风险级别建议：

risk_level	示例	是否可自动 apply
low	alias、tag、observation link、typo	可选
medium	create event、create finance、metadata update	默认人工
high	impact edge、mapping edge、merge、archive	必须人工

9. UI 设计

取消内置 Agent 后，AgentMonitor 可以从四栏改成三栏：

总览

外部 Agent

EventGraph 变更

审计日志

如果暂时不大改 UI，也可以保留“内置 Agent”页签但改名为：

EventGraph 变更

该页显示：

Change Requests
Pending Review
Validation Errors
Recently Applied
Rejected / Needs Changes

每条 change request 显示：

request_id
requester
workflow_id
change_type
risk_level
status
target refs
evidence summary
diff preview
run_id
created_at

操作：

查看 diff
查看 evidence
查看 run trace
批准
拒绝
要求修改
应用

EventGraph 详情页增加 Trace Card：

Origin Request
Origin Run
Actor
Review
Applied Snapshot
Version History

10. 外部 Agent 提示词/契约

外部 agent 在修改 EventGraph 前必须遵守：

1. 先查询正式图谱，避免重复创建。
2. 查询 `derived preview` 和 `observations` 作为证据。
3. 每个 `change request` 只表达一个主目的。
4. `patch item` 必须带 `evidence_refs` 或 `evidence_summary`。
5. 不确定时提交 `low confidence proposal`，而不是直接 `apply`。
6. 不把策略交易判断写成事实 `Edge`。
7. 不把新闻标题直接写成 `verified event`。
8. 不把 `correlation` 写成 `causation`。

推荐 `agent_report` 结构：

```
{
  "what_changed": "新增/修改了什么",
  "why": "为什么需要改",
  "evidence": "证据摘要",
  "uncertainty": "不确定性和替代解释",
  "review_focus": "人类审核重点",
  "strategy_impact": "若用于策略，可能影响哪些策略输入"
}
```

11. 从当前实现迁移

P0: 明确取消内置 Agent

任务：

- 文档中标记 `INT_EVENT_INVESTIGATION` 废弃。
- 前端把“内置 Agent”改为“EventGraph 变更”。
- `_agent_kind()` 可继续兼容 `internal_agent`，但新功能不再新增 `internal workflow`。

P1: 强化 Change Request

任务：

- 给 `agent_event_change_requests` 补字段。

- POST /api/agent/event-graph/change-requests 保存
run_id / workflow_id / patch_json / evidence_summary / risk_level 。
- GET change-requests 解析 payload_json ，返回结构化字段。

P2: Patch Validate

任务：

- 新增 /api/agent/event-graph/patches/validate 。
- 实现 relation type、node type、risk level 校验。
- 前端提交前展示 validation result。

P3: Review UI

任务：

- AgentMonitor 新增 EventGraph 变更页。
- 展示 pending / approved / rejected。
- 支持查看 diff 和 run trace。

P4: Graph Core

任务：

- 新增正式 graph tables。
- change request apply 写入 Graph Core。
- 保留 snapshot 和 version。

P5: Trusted Low Risk Apply

任务：

- 只开放给显式授权 agent。
- 只允许 allowlist action。
- 自动 snapshot 和 audit。

12. 结论

取消内置 agent 后，EventGraph 的设计更清楚：

系统负责可靠性

外部 **agent** 负责智能研究

人类负责最终确认

EventGraph 负责版本化知识

新主线是：

external agent reads EventGraph

-> external agent submits patch/change request

-> system validates

-> human or trusted rule approves

-> system applies versioned graph change

这样既保留了外部 agent 的灵活性，又不会让任何 agent 绕过审计、校验和回滚机制直接污染正式知识库。